

PROJETO INTERDISCIPLINAR

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

Anna Paula Alves Silva	-	26029435
Dilly Martins da Silva	-	26029138
Laura Rayssa Souza Araujo	-	26029075
Matheus Gajewski de Melo	-	26029468
Rafaela Carvalho Barcos Mello	-	26029304

São Paulo

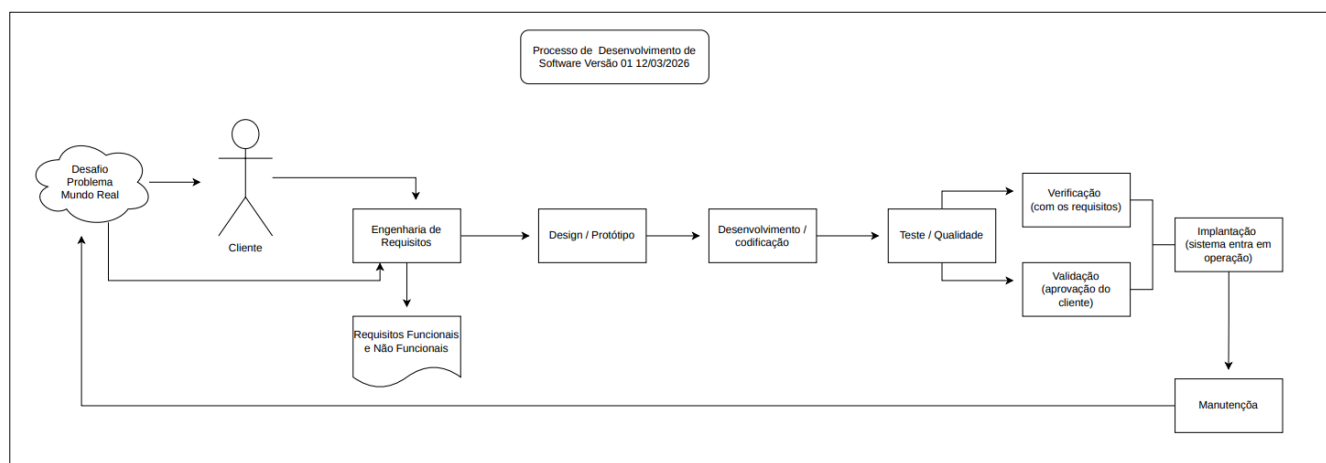
2026

Sumário

1 INTRODUÇÃO	3
1.1 – Processo de desenvolvimento de Software	3
1.2 Apresentação	3
2. DESIGN SPRINT – Ideação e prototipação do desafio	4
2.1 Desafio	4
2.2 Entender Mapear	4
2.2.1 – Entendimento e observação. COMO PODEMOS?	4
2.2.2 – Pesquisa Desk	4
Gcompris: apresentação do jogo - descrição do jogo, pré-requisito (matéria utilizada exemplo, adição), objetivo (o que o aluno vai aprender), manual.	4
2.3 Ideação – desenho da solução (trilha do usuário)	5
2.4 Prototipagem	5
3. REQUISITOS DE SISTEMA	6
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	6
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE	9
4. DIAGRAMAS DE UML	11
4.1 CASOS DE USO	11
Caso de Uso - Generalizado	11
4.2 DIAGRAMA DE CLASSE	12
5. Especificação estrutura de uma rotina do sistema	12
5.1 – Apresentar a especificação de acordo com o modelo abaixo	12
5.2 – Apresentar a codificação da rotina	13
6. REFERÊNCIAS BIBLIOGRÁFICAS	14

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produtos e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - As equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

A segurança do sistema será baseada na validação de rede, onde o acesso é permitido exclusivamente através do endereço IP da instituição de ensino; caso o usuário tente se conectar de uma rede externa, a entrada será automaticamente negada. A verificação será realizada através da conferência de IP de rede, no qual o único autorizado sendo o da escola.

Para garantir a precisão dos dados de uso e diferenciar acessos reais de entradas acidentais, visto que existe um limite para quantos acessos são permitidos, o sistema implementará um rastreamento de fluxo via banco de dados: cada login iniciará uma contagem e somente após esse período de permanência ininterrupta, a sessão será oficialmente contabilizada e registrada no histórico do usuário.

2.2.2 – Pesquisa Desk

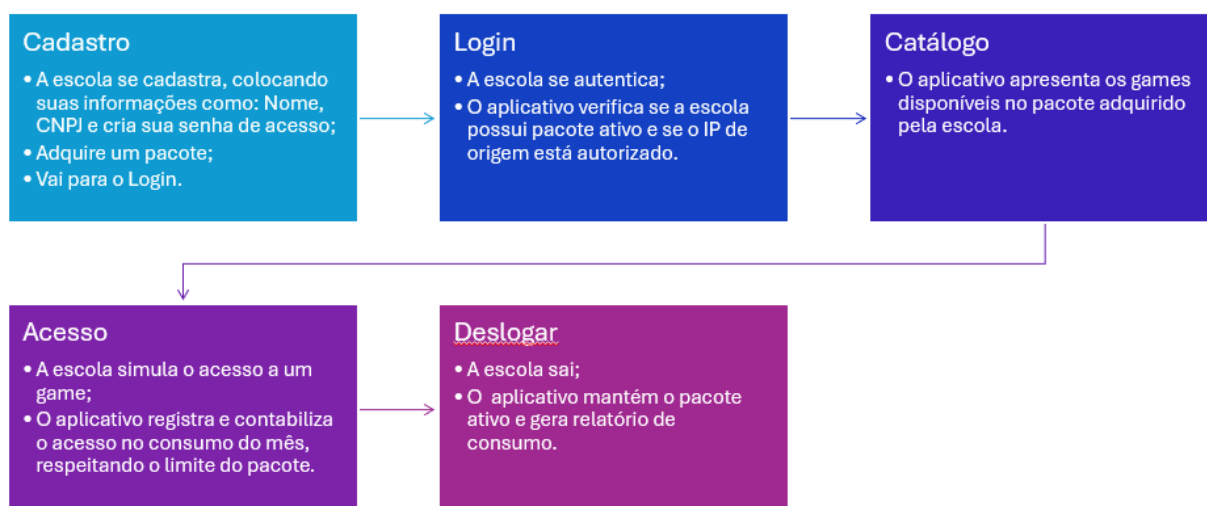
Gcompris: apresentação do jogo - descrição do jogo, pré-requisito (matéria utilizada exemplo, adição), objetivo (o que o aluno vai aprender), manual.

Escola games: carrossel dos jogos.

Nube: interface de login e informações diversas.

2.3 Ideação – desenho da solução (trilha do usuário)

Fluxo do processo (simulação do módulo Escola):



2.4 Prototipagem



3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01	
Função	Cadastro_Games
Descrição	Cadastro de Games (CRUD) com nome, descrição, tema/disciplina, faixa etária e status ativo/inativo;
Entradas	Nome do game, descrição, tema/disciplina, faixa etária, status (ativo/inativo).
Fonte	Administrador do Sistema (Módulo Messier).
Saídas	Confirmação de salvamento/alteração e listagem de games atualizada.
Ação	O sistema deve permitir criar, ler, atualizar e desativar (CRUD) os jogos que compõem os pacotes.

RFS02	
Função	Cadastro_Pacotes
Descrição	Cadastro de Pacotes (CRUD) com: nome do pacote, lista de games incluídos, limite de acessos/mês e preço mensal;
Entradas	Nome do pacote, seleção de games (lista), limite de acessos mensais, preço mensal.
Fonte	Administrador do Sistema.
Saídas	Pacote registrado com vínculo aos games selecionados.
Ação	O sistema deve agrupar games e definir regras de consumo (limites) para comercialização.

RFS03	
Função	Cadastro_Escolas
Descrição	Cadastro de Escolas (CRUD) com dados básicos, pacote adquirido (apenas um pacote ativo por escola) e lista de IPs autorizados;
Entradas	Dados básicos (Nome/CNPJ), seleção do pacote adquirido, lista de IPs autorizados.
Fonte	Usuário Escola
Saídas	Escola cadastrada com pacote ativo e permissões de rede configuradas.

Ação	O sistema deve vincular a escola a um plano de assinatura e validar quais IPs podem acessar o sistema.
-------------	--

RFS04	
Função	Login Escola
Descrição	Login do módulo Escola com validação de pacote ativo e verificação do IP de origem (informado na simulação);
Entradas	Identificação da escola, IP de origem
Fonte	Usuário Escola / Sistema de Rede.
Saídas	Permissão de acesso concedida ou mensagem de erro (IP inválido ou pacote inativo).
Ação	O sistema valida se a escola tem pacote vigente e se o IP atual consta na lista autorizada

RFS05	
Função	Games Disponíveis
Descrição	Exibição dos games disponíveis conforme o pacote ativo da escola;
Entradas	ID da Escola autenticada.
Fonte	Banco de Dados
Saídas	Lista visual dos games que pertencem ao pacote daquela escola.
Ação	O sistema filtra e exibe apenas os jogos que a escola contratou através do seu pacote ativo.

RFS06	
Função	Games_Registros
Descrição	Registro de acesso a game (log com data/hora, escola, game, IP, resultado permitido/bloqueado) e incremento do contador mensal;
Entradas	ID da escola, ID do game, Data/Hora atual, IP de origem.
Fonte	Evento de clique no game / Sistema.
Saídas	Log de acesso gravado e incremento no contador de uso mensal.
Ação	O sistema registra cada tentativa de uso, seja ela permitida ou bloqueada, para fins de auditoria e cobrança

RFS07	
Função	Bloqueio Acesso
Descrição	Bloqueio de acesso quando o limite mensal do pacote for atingido, com mensagem explicativa;
Entradas	Contador de acessos atual, Limite do pacote da escola.
Fonte	Regras de Negócio / Banco de Dados.
Saídas	Mensagem explicativa de bloqueio (ex: "Limite mensal atingido").
Ação	O sistema impede a abertura de novos games caso o consumo do mês tenha chegado ao teto contratado.

RFS08	
Função	Relatorio Acesso
Descrição	Relatório de acessos por escola e por período (ex.: mês/ano), incluindo consumo acumulado;
Entradas	Filtro de período (mês/ano), Identificação da escola.
Fonte	Tabela de Logs/Registros de Acesso.
Saídas	Relatório detalhado com consumo acumulado e histórico de acessos.
Ação	O sistema recupera os dados históricos para demonstrar o uso da plataforma por uma escola específica.

RFS09	
Função	Relatorio Pacotes
Descrição	Relatório de escolas e pacotes (pacote vigente, limite mensal, consumo do mês e percentual de uso).
Entradas	Parâmetros de consulta (todas as escolas ou por pacote).
Fonte	Banco de Dados (Cruzamento de Escolas, Pacotes e Consumo).
Saídas	Lista com: pacote vigente, limite mensal, consumo atual e % de uso.
Ação	O sistema gera uma visão gerencial para o administrador controlar quais escolas estão próximas do limite ou com subutilização

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

RFS01	
Função	Aplicacao Off
Descrição	A aplicação deve funcionar offline (modo local) para fins de protótipo;
Entradas	N/A (Configuração de ambiente local).
Fonte	Documentação de Requisitos / Desenvolvedor.
Saídas	Sistema operacional sem dependência de conexão externa (Cloud/API).
Ação	Garantir que o banco de dados e a aplicação residam na mesma máquina ou rede local, permitindo a simulação completa do CRUD.

RFS02	
Função	Interface Desktop
Descrição	Interface intuitiva e compatível com desktop (Windows), com telas de lista + formulário;
Entradas	Eventos de mouse e teclado (interação do usuário).
Fonte	Bibliotecas de UI
Saídas	Telas responsivas com componentes visuais padronizados.
Ação	Implementar o padrão "Listagem + Cadastro" para facilitar a navegação do usuário e a manutenção do código visual.

RFS03	
Função	Tempo Adequado
Descrição	Tempo de resposta adequado para consultas e relatórios com base de dados pequena/média;
Entradas	Consultas SQL / Filtros de busca.
Fonte	Banco de Dados Relacional.
Saídas	Exibição de dados em tempo real (geralmente abaixo de 2 segundos).
Ação	Otimizar as queries e índices do banco para garantir que o usuário não perceba lentidão ao gerar os relatórios de consumo.

RFS04	
Função	Código Camadas
Descrição	Código modular e organizado em camadas (UI, regras de negócio, dados/persistência);
Entradas	Estrutura de pacotes/pastas do projeto.
Fonte	Padrões de Projeto (ex: MVC ou Layered Architecture).
Saídas	Código-fonte legível e de fácil manutenção.
Ação	Separar a lógica de validação (ex: regra de bloqueio por limite) da lógica de exibição (telas) e do acesso ao banco de dados.

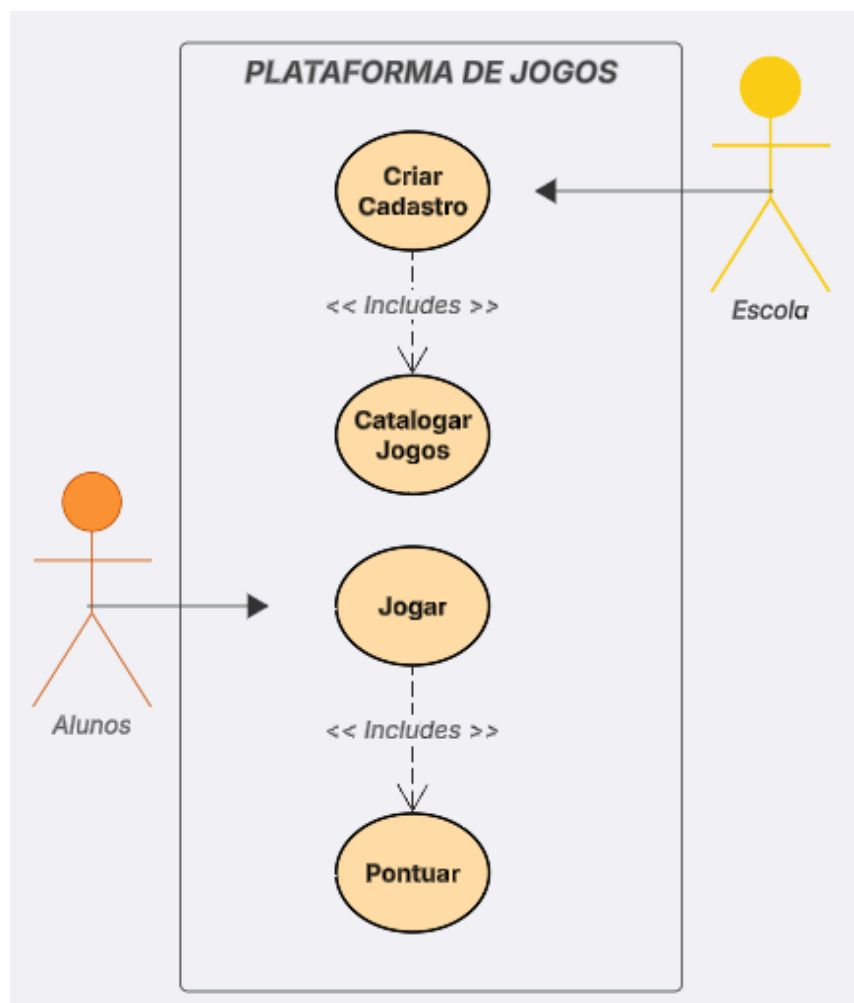
RFS05	
Função	Banco Relacional
Descrição	Persistência em banco relacional com integridade (chaves, relacionamentos e validações);
Entradas	Comandos DDL e DML (Scripts SQL).
Fonte	SGDB (SQLite).
Saídas	Banco de dados estruturado com tabelas relacionadas (Escola-Pacote-Game).
Ação	Implementar chaves primárias e estrangeiras para garantir que, se um pacote for excluído, a integridade dos dados das escolas seja mantida.

RFS06	
Função	Documento_Codigo
Descrição	Documentação clara e padronizada do projeto e do código (README e manual rápido de execução).
Entradas	Informações técnicas do projeto e instruções de build.
Fonte	Desenvolvedor / Padrões Markdown.
Saídas	Arquivos README.md e manuais de usuário/instalação.
Ação	Descrever os passos necessários para configurar o banco de dados e rodar a aplicação pela primeira vez.

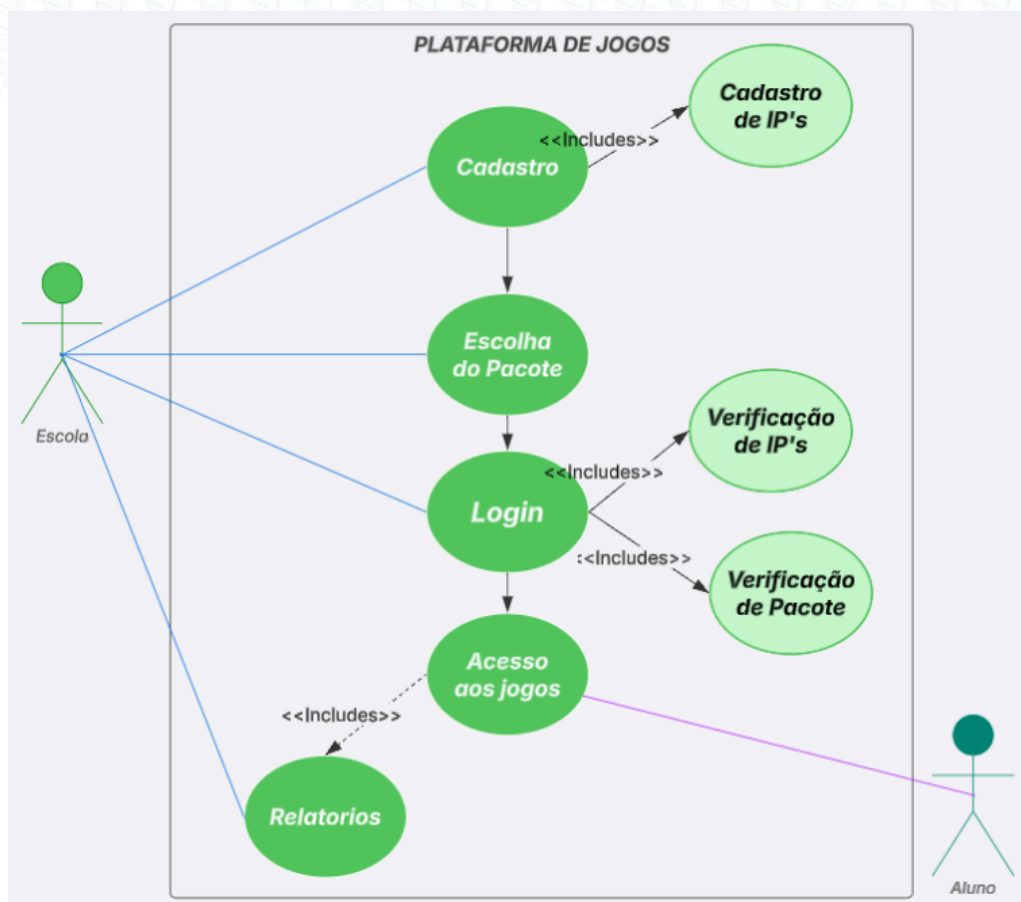
4. DIAGRAMAS DE UML

4.1 CASOS DE USO

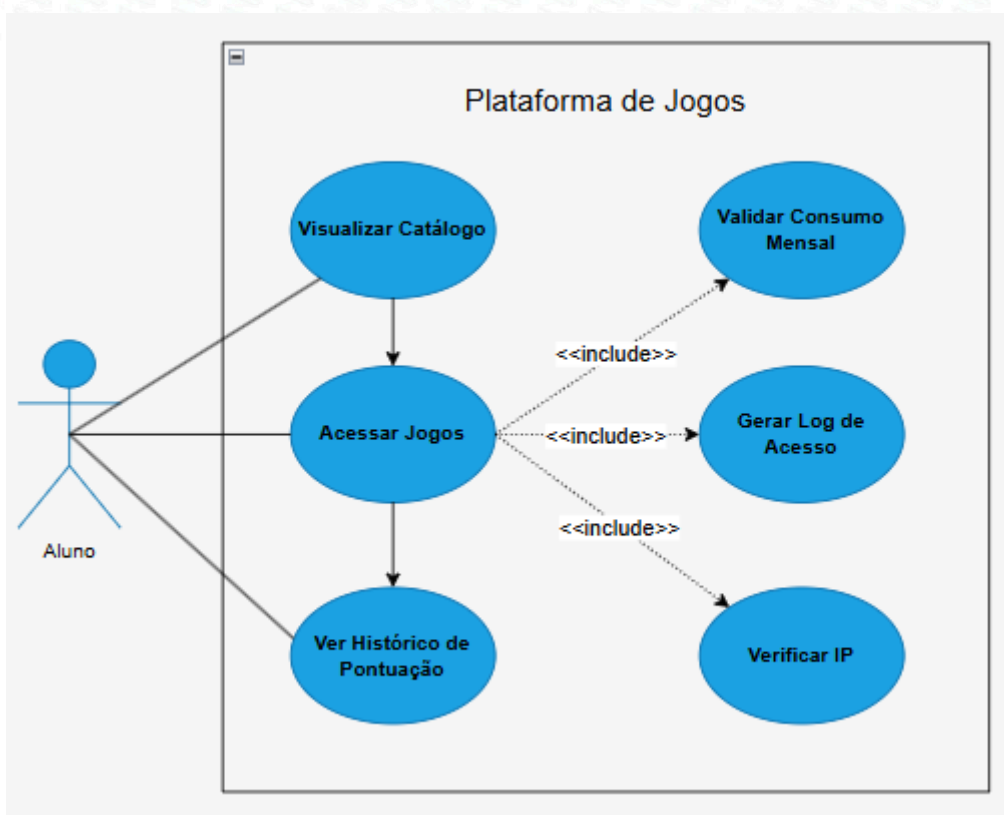
Caso de Uso - Generalizado



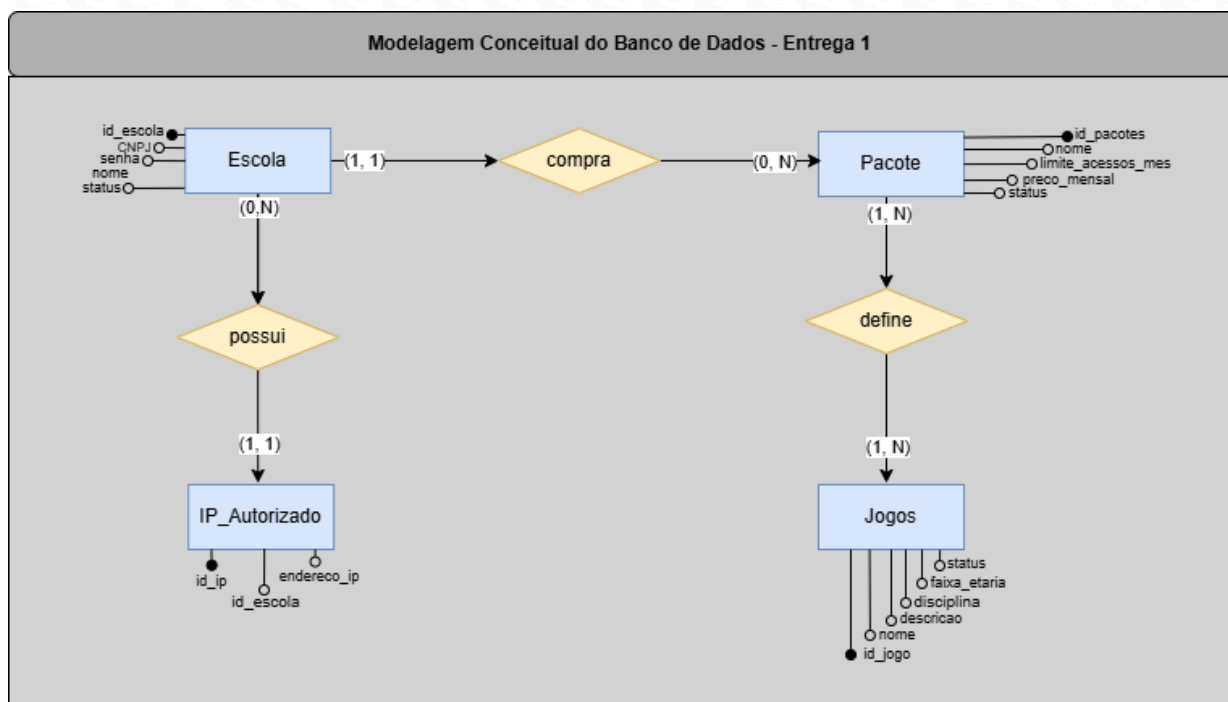
Caso de Uso - Escola



Caso de Uso - Aluno



4.2 DIAGRAMA DE CLASSE



5. Especificação estrutura de uma rotina do sistema

Sistema/Software:

Módulo de Acesso a Games (Plataforma Messier).

Função:

Controlar e registrar acesso a games conforme regras do pacote.

Descrição:

Esta rotina é o "coração" da segurança e do modelo de negócio do sistema. Ela valida se a escola tem direito ao jogo solicitado, se o acesso está sendo feito de um local autorizado (IP) e se o limite de acessos mensais ainda permite a execução do game.

Entradas

- ID da Escola • ID do Game • IP de Origem • Data/Hora da Requisição.

Fonte

- Tabela de Assinaturas/Pacotes • Tabela de Composição de Pacotes • Contador de Acessos Mensais • Tabela de Cadastro de IPs.

Saídas:

- StatusAcesso: Variável booleana ou string (Permitido / Bloqueado) • Registro de log • Mensagem Explicativa: Feedback para o usuário (Ex: "Limite mensal atingido" ou "IP não autorizado").

Destino:

Sistema de execução do game + base de logs.

Ação:

- Verificar se o game pertence ao pacote da escola • Verificar se o limite mensal foi atingido. o Se atingido → bloquear acesso o Caso contrário → permitir acesso • Registrar log com: escola, game, IP, data/hora, resultado • Se permitido → incrementar o contador mensal.

Requisitos:

- Pacote ativo deve estar associado à escola • Limite mensal deve ser definido no pacote • Sistema deve manter histórico de acessos.

Pré-condição:

Escola autenticada no sistema • Conexão ativa com o banco de dados local ou remoto.

Pós-condições:

Log registrado • Contador atualizado (se acesso permitido) • Game iniciado ou mensagem de bloqueio exibida na tela.

Efeitos colaterais:

Geração de dados para o relatório financeiro/gerencial da Messier.

5.2 – Apresentar a codificação da rotina

```
using System;
using System.Collections.Generic;

class Program
{
    // Variáveis Globais (Static para acesso nos métodos)
    static string cnpjCad, senhaCad, nomeEscola, nomePacote = "INVALIDO";
    static List<string> listaIPs = new List<string>();
    static int limiteAcessos, totalConsumido = 0, qtdIPs;
    static bool logado = false;

    static void Main()
    {
        Console.WriteLine("Seja bem-vindo ao Lumina Learning™");
        Console.Write("Possui cadastro? (S/N): ");
        string decisao = Console.ReadLine().ToUpper();

        if (decisao == "S")
        {
            logado = RealizarLogin(cnpjCad, senhaCad);
        }
        else
        {
            // Chamada por referência usando 'ref'
            RealizarCadastro(ref cnpjCad, ref senhaCad, ref nomeEscola, ref
nomePacote, ref limiteAcessos);

            Console.Write("\nDeseja logar agora? (S/N): ");
            if (Console.ReadLine().ToUpper() == "S")
            {
                logado = RealizarLogin(cnpjCad, senhaCad);
            }
        }
    }

    // Módulo de Acesso aos Jogos
    if (logado)
    {

```

```

Console.WriteLine("Deseja abrir um jogo do catálogo? (S/N): ");
string tentativa = Console.ReadLine().ToUpper();

while (tentativa == "S" && totalConsumido < limiteAcessos)
{
    totalConsumido++;
    Console.WriteLine($"Jogo iniciado... Consumo: {totalConsumido} de
{limiteAcessos}");

    if (totalConsumido < limiteAcessos)
    {
        Console.WriteLine("Próximo jogo? (S/N): ");
        tentativa = Console.ReadLine().ToUpper();
    }
    else
    {
        Console.WriteLine($"Limite do pacote {nomePacote} atingido!");
    }
}

Console.WriteLine("--- SESSÃO FINALIZADA ---");
}

// --- MÓDULO 1: CADASTRO ---
static void RealizarCadastro(ref string cnpj, ref string senha, ref string escola, ref
string nPacote, ref int limite)
{
    Console.WriteLine("\n--- CADASTRO ADMINISTRATIVO LUMINA LEARNING
---");
    Console.WriteLine("CNPJ da escola: "); cnpj = Console.ReadLine();
    Console.WriteLine("Nome da escola: "); escola = Console.ReadLine();
    Console.WriteLine("Senha: "); senha = Console.ReadLine();

    Console.WriteLine("Quantos IPs deseja autorizar? ");
    qtdIPs = int.Parse(Console.ReadLine());

    for (int i = 0; i < qtdIPs; i++)
    {
        Console.WriteLine($"Digite o IP {i + 1}: ");
        listaIPs.Add(Console.ReadLine());
    }
}

```

```
// Validação de Pacote com loop
while (nPacote == "INVALIDO")
{
    Console.WriteLine("Escolha o pacote: (1)Bronze (2)Prata (3)Ouro
(4)Diamante");
    string opcao = Console.ReadLine();

    switch (opcao)
    {
        case "1": nPacote = "Bronze"; limite = 100; break;
        case "2": nPacote = "Prata"; limite = 300; break;
        case "3": nPacote = "Ouro"; limite = 400; break;
        case "4": nPacote = "Diamante"; limite = 1000; break;
        default:
            nPacote = "INVALIDO";
            Console.WriteLine("Erro: Opção inválida! Escolha de 1 a 4.");
            break;
    }
}
Console.WriteLine($"Escola cadastrada com sucesso no pacote {nPacote}!");
}

// --- MÓDULO 2: LOGIN ---
static bool RealizarLogin(string c_cad, string s_cad)
{
    for (int tentativas = 1; tentativas <= 3; tentativas++)
    {
        Console.WriteLine($"n--- LOGIN (Tentativa {tentativas}/3) ---");
        Console.Write("CNPJ: "); string c_log = Console.ReadLine();
        Console.Write("Senha: "); string s_log = Console.ReadLine();
        Console.Write("IP Local: "); string ip_log = Console.ReadLine();

        // Verificação de IP no Vetor (Lista)
        bool ipValido = listaIPs.Contains(ip_log);

        if (c_log == c_cad && s_log == s_cad && ipValido)
        {
            Console.WriteLine("Acesso liberado!");
            return true;
        }
        else
    }
```



```
{  
    Console.WriteLine("Dados inválidos ou IP não autorizado!");  
}  
}  
return false;  
}  
}
```

6. REFERÊNCIAS BIBLIOGRÁFICAS

GCompris: GCOMPRIS. **GCompris Educational Software**. Disponível em: https://www.gcompris.net/index-pt_BR.html. Acesso em: 9 maio 2024.

Escola Games: **ESCOLA GAMES. Jogos Educativos**. Disponível em: <https://www.escolagames.com.br/>. Acesso em: 9 maio 2024.

Nube: **NUBE. Núcleo Brasileiro de Estágios**. Disponível em: <https://www.nube.com.br/>. Acesso em: 9 maio 2024.